貼上的文字.txt
TXT

這是我的作業，給我執行步驟，還有每個步驟有需要做甚麼式的細節、promte告訴我，老師打得太複雜了

規劃簡化版的分步執行指南。

作業其實只要做 4 件事：**用 ChatGPT 幫你寫程式跑出結果 → 用 NotebookLM 寫摘要 → 把對話匯出 PDF → 寫報告打包**。下面照順序給你步驟和可以直接複製的 prompt。

步驟 0：先準備好（約 15 分鐘）

1. **註冊 Kaggle 帳號**，資料集選 Playground Series S4E5 "Regression with a Flood Prediction Dataset"（就是補充說的 Predict the Flood）。
   - 20 個數值特徵，目標是 `FloodProbability`。
   - 全部都是數字，不用處理類別編碼，最省事。
   - 它是競賽題，可以上傳結果拿到分數，符合評分項目的「Kaggle 名次」。
2. 進入競賽頁 → **Data** → 下載 `train.csv`、`test.csv`、`sample_submission.csv`。記下網址，報告要寫來源。
3. 開一個 Google Colab 筆記本，命名 `7114056XXX_hw2.ipynb`（換成你的學號）。
4. 開一個**新的 ChatGPT 對話**，整份作業都在這個對話裡問，最後才好一次匯出。

步驟 1：用 ChatGPT 寫程式（依 CRISP-DM 分段問）

每段 prompt 問完，把程式貼到 Colab 執行。有錯誤就把錯誤訊息貼回 ChatGPT 修。

Prompt 1：Business + Data Understanding

```
我要用 Kaggle 的 Playground Series S4E5 Flood Prediction 資料集做多元線性回歸作業，
需遵循 CRISP-DM 流程。目標變數是 FloodProbability，有 20 個數值特徵。
請幫我寫 Google Colab 的 Python 程式：
1. 用一段文字說明 Business Understanding（預測洪水機率的商業/社會價值）
2. 讀取 train.csv，顯示資料筆數、欄位、資料型態、缺失值、描述統計
3. 畫出目標變數分布圖、特徵相關係數熱力圖、各特徵與目標的相關係數長條圖
請加上中文註解，圖表標題用英文避免字型問題。
```

Prompt 2：Data Preparation

```
接續上面，請幫我做 Data Preparation：
1. 刪除 id 欄位，檢查並處理缺失值與重複值
2. 用 train_test_split 切成 80% 訓練、20% 測試（random_state=42）
3. 用 StandardScaler 標準化（只用訓練集 fit）
4. 簡單說明每一步為什麼要這樣做
```

Prompt 3：Feature Selection（必做，最重要）

```
請幫我做特徵選擇，比較三種方法，只用訓練資料：
1. SelectKBest (f_regression)
2. RFE（搭配 LinearRegression）
3. LassoCV（看哪些係數變成 0）
分別選出 k = 5, 10, 15, 20 個特徵，用 5-fold 交叉驗證比較 R²，
畫出「特徵數 vs R²」折線圖，並用表格列出每個方法選到的特徵。
最後建議選哪一組特徵並說明理由。
```

Prompt 4：Modeling

```
用選出的特徵建立模型並比較：
1. LinearRegression（sklearn）
2. Ridge、Lasso
3. 用 statsmodels OLS 再建一次，印出 summary（係數、p-value、R²、Adjusted R²）
並解釋前 5 個最重要特徵的係數意義。
```

Prompt 5：Evaluation + 預測區間圖（老師要求必有）

```
請幫我做模型評估：
1. 在測試集計算 MAE、MSE、RMSE、R²、Adjusted R²，整理成比較表
2. 畫「實際值 vs 預測值」散佈圖，加上 y=x 參考線
3. 用 statsmodels 的 get_prediction().summary_frame(alpha=0.05) 取得
   95% 信賴區間 (mean_ci) 與 95% 預測區間 (obs_ci)，
   抽 100 筆測試資料依預測值排序，畫出預測線 + 信賴區間 + 預測區間陰影 + 實際值點
4. 殘差分析：殘差 vs 預測值圖、殘差直方圖、Q-Q plot、計算 VIF 檢查共線性
   並說明是否符合線性回歸的假設
資料有一百多萬筆，畫圖時請抽樣避免太慢。
```

Prompt 6：Deployment + Kaggle 提交

```
請幫我：
1. 用全部訓練資料重新訓練最終模型，對 test.csv 預測，輸出 submission.csv
2. 用 joblib 儲存模型
3. 寫一個 predict_flood(輸入資料) 函式示範如何預測新資料
4. 用一段文字說明這個模型實際上可以怎麼部署使用
```

拿到 `submission.csv` 後：到競賽頁按 **Submit Prediction** 上傳（比賽已結束仍可 Late Submission），**截圖分數**放報告裡。線性回歸大約能拿到 R² 0.84～0.85 左右。

Prompt 7：比較網路更優解（報告要用）

```
這個競賽網路上主流的高分做法有哪些？（例如把特徵加總、統計特徵、
LightGBM/XGBoost、集成模型）請和我的線性回歸結果比較，
用表格整理優缺點，並說明為什麼線性回歸在這題表現已經不錯。
```

（ChatGPT 的說法要自己對照 NotebookLM 查到的資料，不要全信。）

步驟 2：NotebookLM 研究摘要

1. 打開 notebooklm.google.com，建立新筆記本。
2. **新增來源**（貼網址）：到 Kaggle 競賽頁的 **Code** 和 **Discussion** 分頁，挑 3～5 篇高票的 notebook 或討論，再加 1～2 篇介紹多元線性回歸/特徵選擇的文章。
3. 在 NotebookLM 輸入：

```
請根據這些來源，用繁體中文寫一份約 200 字的摘要，內容包含：
1. 大家解這題常用的方法與模型
2. 常用的特徵工程或特徵選擇技巧
3. 表現最好的做法及其分數
4. 線性回歸在這題的優缺點
```

4. 把回答**截圖** + 複製文字，報告中標題寫「**NotebookLM 摘要**」。

步驟 3：匯出 ChatGPT 對話為 PDF

- 方法 A：安裝 pdfCrowd 的 Chrome 擴充功能，在對話頁面一鍵存 PDF。
- 方法 B（更簡單）：在對話頁按 `Ctrl+P` → 目的地選「另存為 PDF」。

檔名例如 `GPT_conversation.pdf`。

步驟 4：寫報告（PDF）

照這個目錄寫，每節放對應的圖和說明：

1. **資料集來源**：名稱、Kaggle 網址、筆數、20 個特徵是什麼
2. **Business Understanding**：為什麼預測洪水機率有用
3. **Data Understanding**：描述統計、分布圖、相關熱力圖
4. **Data Preparation**：缺失值、切分、標準化
5. **Modeling**：特徵選擇比較圖表 + 選擇理由、模型係數解讀
6. **Evaluation**：指標比較表、實際 vs 預測圖、**預測區間圖**、殘差圖、Kaggle 分數截圖
7. **Deployment**：如何使用模型
8. **網路主流解法比較**：表格（你的線性回歸 vs 特徵加總技巧 vs 樹模型）
9. **NotebookLM 摘要**（標題要寫清楚）
10. **GPT 輔助內容**：說明哪些部分用 GPT 協助，並附上對話 PDF（可合併到報告最後，或另附檔）

可以用 Word/Google 文件寫，最後匯出成 PDF。

步驟 5：打包繳交

```
7114056XXX_hw2.zip
├── 7114056XXX_hw2.ipynb       ← 主程式（記得執行過、有輸出結果）
├── 7114056XXX_hw2_report.pdf ← 報告
├── GPT_conversation.pdf       ← 若沒合併進報告
└── （選做）README.md + GitHub/Colab 連結
```

建議順序與時間：步驟 0～1 最花時間（約 3～4 小時），2～5 約 2 小時。程式卡住的話，把錯誤訊息貼給我也可以；需要的話我也能直接幫你寫好完整的 `.ipynb` 或報告架構。

5115056015_hw2.ipynb
IPYNB

Flood_Prediction_CRISP_DM_Report.docx
DOCX

PDF

PDF

一、作業目標：
本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進行「多元線性回歸 (Multiple Linear Regression)」的完整分析，並遵循 CRISP-DM 流程完成從資料理解、建模到評估的全過程。

二、作業內容：
1. 資料來源
 至 Kaggle 選擇一個具有 10 至 20 個特徵 (features) 的公開資料集。
 類型不限（可為房價預測、醫療、車輛效能等主題）。

顯示更多

讀取了一個檔案、執行了 5 個指令 ▸

我把四個檔案都看過了，也對照了評分標準。程式和分析本身做得很好：CRISP-DM 六步都有，特徵選擇比較了三種方法，預測區間圖、殘差、VIF 都有，Kaggle 分數也有。真正會扣分的是**報告缺兩個指定標題**，另外還有幾個小地方要修。

🔴 一定要補（不補會直接扣分）

1. 報告沒有「NotebookLM 摘要」這一節（屬於 15% 那項）
你已經做了 NotebookLM，但報告裡沒放。在第 8 節後面加一節，標題要一字不差，可以直接貼：

> 9. NotebookLM 摘要
> 本研究使用 NotebookLM（Gemini Notebook）匯入 7 篇來源，包括 Kaggle 競賽頁、高分 Notebook 與討論串，研究網路上的解法。摘要如下：
> 參賽者大多用梯度提升樹（CatBoost、LightGBM、XGBoost），再搭配線性模型（Linear Regression、Ridge）做模型融合（Blending/Ensemble）。拿高分的關鍵是特徵工程，最核心的做法是對每一筆資料計算 20 個特徵的列統計量（Row-wise statistics），例如 sum、mean、std、中位數、skew、entropy。加了這些特徵後，CatBoost 單一模型的 CV R² 達 0.87003，LightGBM 約 0.86931。線性回歸的優點是訓練非常快，只用特徵總和就能拿到 0.844～0.846 的基準分數；缺點是沒辦法捕捉非線性關係和特徵交互作用，所以上限比較低。
> （附上 NotebookLM 畫面截圖）

2. 報告沒有「GPT 輔助內容」這一節
加一節，說明哪些部分用了 GPT，並註明對話 PDF 附在報告後面：

> 10. GPT 輔助內容
> 本作業使用 ChatGPT 協助以下部分，完整對話見附錄（對話 PDF，共 61 頁）：
>
> | 階段 | GPT 協助內容 | 我自己做的部分 |
> |---|---|---|
> | Business / Data Understanding | 產生 EDA 程式與說明文字 | 在 Colab 執行、解讀結果 |
> | Data Preparation | 前處理程式碼（切分、標準化） | 執行並確認沒有資料洩漏 |
> | Feature Selection | 比較三種方法的 CV 程式 | 依結果決定保留 20 個特徵 |
> | Modeling / Evaluation | OLS、預測區間、殘差、VIF 程式 | 修正 get_prediction 錯誤、解讀診斷結果 |
> | Deployment | submission、joblib、predict_flood | 上傳 Kaggle 取得分數 |
> | 網路解法比較、報告 | 整理高分解法、排版 Word 報告 | 對照 NotebookLM 來源查證 |

3. 報告要交 PDF，不是 Word
Word 用「另存為 PDF」，再把 GPT 對話 PDF 接在報告最後面。用 ilovepdf 之類的工具合併就可以。

4. 刪掉看起來像 AI 寫的句子

- 封面那句「資料來源：使用者提供之 Google Colab Notebook…及 Kaggle Submission 截圖…」整句刪掉。
- 圖 13 說明裡的「（使用者提供原始截圖）」刪掉。
- 參考資料最後兩條「Original user Notebook」「User Kaggle Submission screenshot…png」刪掉。

5. 封面加上學號、姓名、課程名稱

🟡 程式要修

6. Cell 37 有一個紅色錯誤：`ols_model.get_prediction(...)` 那格出現 `ValueError`。這是貼錯的殘留，下一格才是正確版本。把這格刪掉，最後一格空白的也刪掉。老師打開看到紅字，「模型正確可執行」那項可能被扣分。

7. 在 Notebook 補幾格文字說明：現在幾乎都是程式。在特徵選擇、Evaluation、Deployment 結果後面各加一格 Markdown，寫 2～3 句結論，可以直接從報告抄過來。另外標題編號有點亂（「3. Data Preparation」之後是「4. EDA」，接著又回到 Data Preparation），建議統一照 CRISP-DM 六步編號。

🟢 加分項（衝滿分建議做）

8. Kaggle 名次：評分寫「呈現 Kaggle 名次（若有）」。賽後提交不會給名次，但你可以到 Leaderboard → **Private** 分頁，找出 0.84426 會排在哪裡，在報告寫「Private 0.84426，對照排行榜約落在第 X / 總隊伍數 名（賽後提交，僅供參考）」。

9. 補跑 GPT 建議的「特徵加總」實驗：GPT 在對話 PDF 第 58 頁給過這段程式，但你沒跑。現在三種特徵選擇最後都選全部 20 個特徵，老師可能覺得有點沒選。補跑這個實驗就能說明「1 個加總特徵 ≈ 20 個特徵」，特徵選擇和網路解法比較兩項都會更有說服力。貼到 Notebook 最後一格：

```python
# 額外實驗：只用 1 個特徵（20 個特徵加總）vs 全部 20 個特徵
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
Xtr_sum = X_train.sum(axis=1).to_frame("feature_sum")
Xte_sum = X_test.sum(axis=1).to_frame("feature_sum")
r2_sum = r2_score(y_test, LinearRegression().fit(Xtr_sum, y_train).predict(Xte_sum))
r2_all = r2_score(y_test, LinearRegression().fit(X_train, y_train).predict(X_test))
print(pd.DataFrame({"Features": ["feature_sum (1)", "All 20"], "Test R2": [r2_sum, r2_all]}))
```

跑完把結果表放進報告第 8 節。

10. README.md + Colab 連結（optional，但有寫就是加分）：在 Colab 按「共用」→ 改成「知道連結的人都能檢視」，複製連結。README 寫 5～6 行就夠：資料集、流程、最終 R²、Kaggle 分數、連結。

最後的 zip 內容

```
5115056015_hw2.zip
├── 5115056015_hw2.ipynb          ← 刪掉錯誤格後重新存
├── 5115056015_hw2_report.pdf     ← 含 NotebookLM 摘要、GPT 輔助內容，最後附 GPT 對話
├── GPT_conversation.pdf          ← 單獨再放一份也可以
├── NotebookLM_summary.pdf        ← 你那份 Gemini Notebook PDF
└── README.md                     ← 附 Colab 連結
```

最重要的是第 1～3 項，補完基本上就沒有缺項了。如果需要，我可以直接幫你把第 9、10 節和封面資訊加進 Word、刪掉那幾句，再轉成 PDF；把你的姓名給我就行。

1. 幫我修正5115056015_hw2.ipynb，幫我修正，並給我完整版，我會在自己跑完一次檔案
2. 5115056015_hw2_report幫我新增內容成word，並依照你的建議修正，我在自己轉pdf

按照CRISP-DM流程重新编排标题结构 ▸

我會修好 Notebook（刪錯誤格、整理 CRISP-DM 標題、補結論說明、加特徵加總實驗，讀檔改成可以重跑），再更新 Word 報告（補 NotebookLM 摘要、GPT 輔助內容、附錄對話紀錄，並刪掉那幾句）。

建立了一個檔案、執行了 3 個指令 ▸

Notebook 已重組。接著用一份格式相同的小型模擬資料實際跑一遍，確認每一格都能執行、不會報錯。

建立了一個檔案、已分享 2 個檔案、以及另外 16 個步驟 ▸ +3

兩個檔案都改好了。我用格式相同的模擬資料把 Notebook 從頭到尾跑過一次，沒有任何錯誤。真正的結果要等你用 Kaggle 資料自己跑。

5115056015_hw2.ipynb 改了什麼

- **刪掉**紅色錯誤那格（`get_prediction(...)`）和最後的空白格。
- **開頭加一格**，寫學號、資料集連結、Colab 連結，還有一張 CRISP-DM 章節對照表。
- **標題統一**成 1～6 對應 CRISP-DM 六步，最後加一節「7. 結論」。
- **補了 9 格說明**：每個階段跑完都有 2～4 句結論，殘差診斷整理成表格。
- **新增 5.6 特徵加總實驗**：比較只用 1 個 `feature_sum` 和用全部 20 個特徵的 R²，並畫一張散佈圖。
- **讀檔不用一直手動上傳**：`train.csv` 和 `test.csv` 如果已經在 Colab 資料夾裡會直接讀取，找不到才跳出上傳視窗。

Notebook 開頭有兩個空白（黃字 ＿＿＿）要填：**姓名**和 **Colab 連結**。

5115056015_hw2_report.docx 改了什麼

- 新增 **8.1 特徵加總實驗**、**9. NotebookLM 摘要**（附截圖、提示詞、和你結果的對照）、**10. GPT 輔助內容**（分階段列出 GPT 做了什麼、你做了什麼）。
- **附錄 A**：把 61 頁 GPT 對話直接放進報告，轉 PDF 時會一起轉出去，不用另外合併。
- 刪掉「使用者提供…」那幾句，封面改成學號 / 姓名 / 課程。
- 6.1 節加了 Kaggle 名次那一句；結論和參考資料重新編號。

你要做的（報告裡都標了黃底）

1. 封面：填**姓名、課程、Colab 連結**。
2. 6.1 節：到 Kaggle Leaderboard → **Private**，找出 0.84426 大概排第幾名、總共幾隊，填進去。
3. 8.1 節：跑完 Notebook 後，把 **5.6 節表格 A 列的 MAE、RMSE、R²** 填進表格，再把 5.6 的圖貼上來，取代黃色框框。
4. 填完把**黃底螢光全部清掉**，再存成 PDF（大約 77 頁）。

重跑之後，除了新加的 5.6 節，其他數字應該都和原本一樣，因為 random_state 都固定了。如果哪個數字變了，跟我說，我幫你改報告。

5115056015 hw2
IPYNB

5115056015 hw2 report
文件 · DOCX

下載全部

D:\AIoT_作業
幫我檢查，看缺什麼幫我完成
也幫我詳細看需要打包的文件內容有沒有需要修正，幫我修正，我在自己壓縮成zip

已要求存取您電腦上的資料夾、已列出您電腦上的檔案，以及另外 2 個步驟 ▸

Notebook 從頭到尾都跑完了，沒有錯誤，所有數字也和報告一致。特徵加總實驗的結果是 R² 0.844863，20 個特徵是 0.844877。接下來檢查報告。

讀取了 2 個檔案、搜尋了網路，以及另外 6 個步驟 ▸

我檢查完了，缺的東西也都補好了。Notebook 已經用真資料完整跑過，沒有錯誤；報告裡的數字我一個一個對照過 Notebook 輸出，全部一致。修好的檔案已經直接寫回 D:\AIoT_作業。

這次修了什麼

- **報告 docx**
  - 圖 14 的說明還留著「請以 Notebook 輸出圖替換此框」，已改成正式說明。
  - 8.1 節的結論補上實際數字：只用 1 個加總特徵 R² = 0.844863，20 個特徵是 0.844877，只差 0.000014。
  - 名次那句整理成「約可排在第 1,845 名」。
- **Notebook**
  - 開頭補上姓名、課程、Colab 連結。
  - 5.6 節的結論改成用實際數字寫。
  - Deployment 那節加上 Kaggle 名次。
  - 原本的執行結果都保留，不用重跑。
- **新增 README.md**：報告封面寫了會附 README，但原本資料夾裡沒有。內容有檔案說明、CRISP-DM 流程、成果表和 Colab 連結。

壓縮前你要做的

1. 用 Word 打開 `5115056015_hw2_report.docx`，**重新另存成 PDF**，覆蓋舊的 `5115056015_hw2_report.pdf`。舊 PDF 是修正前轉的。
2. **建議加上總隊伍數**：我查不到這場比賽總共幾隊。名次那句目前只有「第 1,845 名」，你可以到 Leaderboard 看一下，補成「第 1,845 名 / 共 X 隊」，會更完整。

放進 zip（5115056015_hw2.zip）

```
5115056015_hw2.ipynb
5115056015_hw2_report.pdf      ← 重新轉的那份
README.md
GPT_conversation.pdf
NotebookLM_summary.pdf
submission.csv
flood_prediction_model.joblib
```

不要放進去的：`playground-series-s4e5` 資料夾和 `playground-series-s4e5.zip`（Kaggle 原始資料，約 120MB，老師自己可以下載）、`.ipynb_checkpoints` 資料夾、`5115056015_hw2_report.docx`（老師要的是 PDF）。

貼上的文字.txt
TXT

這是加分項
幫我檢查目前有沒有完成"D:\中興碩班\AIoT_作業\HW2"
我也新建了https://github.com/EasyBoy5566/AIoT-HW2.git

Checking the HW2 folder and GitHub repo access.

我先看你電腦上的 HW2 資料夾和 GitHub repo 現在有什麼，再對照加分項要求。

搜尋了過去的對話、已要求存取您電腦上的資料夾，以及另外 2 個步驟 ▸

還沒完成。你資料夾裡的東西只符合舊版要求（Notebook + 報告 + zip）；新版主要繳交的「GitHub + 網頁」和兩個加分項都還沒做，而且 GitHub repo 目前是空的。

對照新版要求

項目	狀態	說明
Kaggle 資料集 > 13 個特徵	✅	Flood Prediction 有 20 個特徵
線性回歸、特徵選擇、模型評估	✅	Notebook 都有
預測圖加信賴區間 / 預測區間	✅	Notebook 5.3 節
CRISP-DM 六步	✅	Notebook 和報告都有
GPT 對話 PDF、NotebookLM 摘要、兩個標示	✅	報告第 9、10 節與附錄 A
GitHub（主要繳交）	❌	repo 是空的，沒有任何 commit
Web（主要繳交）	❌	沒有網頁版
加分 1 分：下拉選單切換兩種 problem	❌	沒做
加分 2 分：可輸入任意 Dataset 網址	❌	沒做
選做：對話內容、資訊圖表、報告書	🟡	對話 PDF 和報告有了，資訊圖表沒有

還要做的事

1. **做一個 Streamlit 網頁**（`app.py`），在網頁上跑完整個 CRISP-DM 流程：
   - 側邊欄**下拉選單**選 problem：① Flood Prediction（洪水機率）② 50 Startups（預測公司 Profit）→ 加分 1 分
   - 另一個選項「自訂 Dataset」：貼任意 CSV 網址（例如老師給的 50_Startups.csv），網頁自動讀檔，讓你選目標欄位，類別欄位自動做 One-hot 編碼 → 加分 2 分
   - 每個資料集都會顯示：資料概覽 → 相關係數熱力圖 → 特徵選擇比較（SelectKBest / RFE / Lasso）→ 模型評估表 → **預測圖加 95% 信賴區間 / 預測區間** → 殘差圖 → 輸入數值即時預測
2. **把檔案推上 GitHub**：`app.py`、`requirements.txt`、Notebook、報告 PDF、README（補上網頁連結和截圖）。
3. **部署到 Streamlit Community Cloud**：用 GitHub 帳號登入 share.streamlit.io，選這個 repo，免費拿到一個網址。這步要你自己點，大約 3 分鐘。
4. （選做）做一張資訊圖表。NotebookLM 的「資訊圖表」功能可以直接產生。

Flood 的 train.csv 有 59MB，網頁上我會改用 5 萬筆抽樣資料（約 3MB）放進 repo，結果和全資料幾乎一樣，網頁也能很快跑完。

如果要，我可以直接幫你寫好 `app.py`、`requirements.txt`、抽樣資料和新的 README，先在這裡實際跑過，再放進你的 HW2 資料夾。推上 GitHub 有兩種方式：你自己用 GitHub Desktop 上傳，或是我透過 GitHub 連線幫你直接 push。要我開始做嗎？要用哪種方式推？

幫我完成

Planning the repository push and data staging ▸

我會寫好 Streamlit 網頁、抽樣資料和 README，先在這裡實際跑過，再放進你的 HW2 資料夾並推上 GitHub。

建立了一個檔案、執行了 9 個指令，以及另外 4 個步驟 ▸

三個選項都能跑完，預測也正常。再測一個有缺失值、文字欄位，又是 GitHub 網頁連結格式的資料集，以及一個錯誤網址。

建立了 2 個檔案、已使用 新增 repo、以及另外 23 個步驟 ▸

加分項的兩個功能都做好了，GitHub repo 也已經上傳完成。只剩**部署網頁**這一步：要用你的 GitHub 帳號登入 Streamlit，所以要你自己操作，大約 3 分鐘。

已完成

- **網頁 `app.py`（Streamlit）**：分成 7 個分頁，照 CRISP-DM 六步排，最後一頁是網路解法、NotebookLM 摘要和 GPT 輔助內容。
  - **+1 分**：側邊欄的下拉選單可以切換「Problem 1：洪水機率預測」和「Problem 2：新創公司利潤預測（50 Startups）」。
  - **+2 分**：選「自訂 Dataset（CSV 網址）」後，可以貼任何 CSV 網址，也可以直接上傳 CSV，再選要預測的欄位。缺失值、文字欄位和 ID 欄位都會自動處理。
  - 每個資料集都會跑出這些結果：三種特徵選擇方法的比較圖和自動推薦、模型比較、**預測圖（含 95% 信賴區間和預測區間）**、殘差圖、Q-Q 圖、VIF，最後可以輸入數值即時預測。
- **測試**：兩個 problem、老師的 50_Startups 網址，再加兩個有缺失值和文字欄位的資料集都跑過了，沒有錯誤。錯誤網址會顯示提示，不會整個網頁當掉。
- **GitHub（EasyBoy5566/AIoT-HW2）**：已上傳網頁程式、Notebook、報告 PDF、GPT 對話 PDF、NotebookLM PDF、模型、抽樣資料，以及附截圖的 README。README 裡有加分項對照表。
- **你的 HW2 資料夾**：也放了同一份 `app.py`、`requirements.txt`、`data/`、`images/`，README 換成新版。

洪水資料在網頁上用的是 5 萬筆抽樣，測試 R² 是 0.848；用完整 111 萬筆資料的結果（0.8449）寫在 Notebook 和報告裡。

你要做的：部署網頁

1. 到 **share.streamlit.io**，用 GitHub 帳號登入。
2. 按 **Create app** → 選「**Deploy a public app from GitHub**」。
3. **Repository** 選 `EasyBoy5566/AIoT-HW2`，**Branch** 填 `main`，**Main file path** 填 `app.py`。
4. **App URL 填** `aiot-hw2-5115056015`。README 已經寫好這個網址，填一樣的就不用再改。
5. 按 **Deploy**，等 2～3 分鐘安裝套件。完成後打開網址，確認下拉選單能切換。

如果那個網址名稱被別人用掉了，換一個也可以，把新網址告訴我，我幫你更新 README。

可以用github pages嗎

Checking which Pyodide packages stlite can load ▸

可以，但沒辦法直接用。GitHub Pages 只能放靜態網頁（HTML、JS），不能執行 Python；Streamlit 需要一台跑 Python 的伺服器，所以 `app.py` 放上去不會動。

不過有一個辦法：用 **stlite**，它會把 Python 直接放在瀏覽器裡跑。同一份 `app.py` 包進一個 `index.html` 就能放上 GitHub Pages，網址會是 `https://easyboy5566.github.io/AIoT-HW2/`。

	Streamlit Cloud（目前做法）	GitHub Pages + stlite
程式要改的地方	不用改	要小改：讀取網址的方式，洪水資料抽樣改小一點
第一次打開	有時要等 30 秒喚醒	每次都要先下載約 30MB 的 Python 套件，大約 20～60 秒
計算速度	快	比較慢，洪水資料的特徵選擇可能要跑十幾秒到一分鐘
自訂 Dataset 網址	任何網址都可以	只能讀允許跨網域讀取（CORS）的網址；raw.githubusercontent.com 可以，所以老師的 50_Startups 範例沒問題
會不會休眠	一段時間沒人用會休眠，要按一下喚醒	不會休眠，永遠在線
部署	要你登入 share.streamlit.io 點幾下	我可以直接幫你推上 GitHub，你只要開一次 Pages 設定

我的建議是**兩個都做**：README 同時放兩個網址，一個打不開還有另一個。如果只能選一個，我選 Streamlit Cloud，比較穩、也比較快；GitHub Pages 的好處是不用另外註冊，也不會休眠。

要我幫你做 GitHub Pages 版本嗎？我會改好，在這裡測過能跑，再推上你的 repo。

做 GitHub Pages 版本

Checking which stlite release is available on npm ▸

我會做一個 `index.html` 用 stlite 載入同一份 `app.py`，讓網頁在瀏覽器裡跑 Python。先在這裡用瀏覽器實測能跑，再推上 GitHub 並開啟 Pages。

讀取了一個頁面、執行了 3 個指令 ▸

stlite 1.12.0 用的是 Streamlit 1.65 和 pandas 3，和我測試的環境很接近。現在修改 `app.py`，讓它在瀏覽器裡也能讀網址，再寫 `index.html`。

建立了 2 個檔案、已使用 Claude in Chrome（27 個動作）、以及另外 28 個 ▸ +14

GitHub Pages 版本已經上線：https://easyboy5566.github.io/AIoT-HW2/

我在你的 Chrome 裡實際打開測過：

- 網頁能正常載入。
- Evaluation 分頁的預測區間圖、殘差圖、Q-Q 圖都畫得出來。
- 下拉選單選「自訂 Dataset（CSV 網址）」後，老師的 50_Startups 網址也能讀進來並跑完。

做了什麼

- `index.html`：用 stlite 1.12.0，讓同一份 `app.py` 直接在瀏覽器裡執行 Python，不需要伺服器。
- `app.py`：
  - 在瀏覽器裡改用另一種方式讀取網址。
  - 洪水資料在瀏覽器版只用 20,000 筆，跑得比較快。
  - 頂端的指標改成會自動換行的卡片。原本筆電寬度下數字會被截成「0.85…」，現在會完整顯示。
- **GitHub**：已推上 repo，並在 Settings → Pages 設定成從 `main` 分支發布。README 的主要網址也換成 GitHub Pages。
- **你的 HW2 資料夾**：也放了同一份新的 `index.html`、`app.py`、README 和截圖。

要知道的幾件事

- **第一次打開要等 20～60 秒**，因為瀏覽器要先下載 Python 環境，頁面頂端有提示。之後切換資料集也要等幾秒。
- **自訂 Dataset 只能讀允許跨網域讀取的網址**。raw.githubusercontent.com 可以，所以老師的範例沒問題。其他網站如果讀不到，網頁會顯示提示，改用「上傳 CSV」就可以。
- 這個版本的洪水資料是 20,000 筆，測試 R² 是 0.854；完整 111 萬筆的結果在 Notebook 和報告裡。
- README 裡還留著一條 Streamlit Cloud 備用網址，但那個還沒部署，目前打不開。只交 GitHub Pages 的話，可以叫我把它刪掉。

繳交時給老師兩個網址就可以：GitHub repo `https://github.com/EasyBoy5566/AIoT-HW2`，和網頁 `https://easyboy5566.github.io/AIoT-HW2/`。

貼上的文字.txt
TXT

在幫我檢查
所有內容有沒有符合需求

Deciding whether to add an infographic ▸

我逐項對照需求，檢查 GitHub repo、網頁和報告，有缺的直接補上。

執行了 3 個指令 ▸

Works. One judgment rule is too strict: with only 10 test samples, 50 Startups gets flagged "偏離 0". I'll switch it to a statistical check (mean within 2 standard errors of 0).

Fix residual-mean rule and retest all options ▸

All three options pass. One optional item is still missing: the 資訊圖表 (infographic). I'll make a one-page infographic summarizing the project and add it to the repo and README.

建立了一個檔案、已使用 Claude in Chrome（8 個動作）、以及另外 5 個 ▸ +2

我逐項檢查過了，**必要項目和兩個加分項全部符合**。檢查時發現三個地方可以更好，已經補上，也推上 GitHub 和網頁了。

對照需求

需求	狀態	在哪裡
加分 +1：下拉選單呈現兩種 problem	✅	網頁側邊欄：洪水機率預測 / 新創公司利潤預測
加分 +2：可任意輸入 Dataset	✅	下拉選「自訂 Dataset」，老師的 50_Startups 網址已在線上版實測可用
Kaggle 資料集、特徵 > 13、標示來源連結	✅	Flood S4E5，20 個特徵；網頁頂端、README、報告都有連結
線性回歸	✅	LinearRegression、Ridge、Lasso、OLS
特徵選擇	✅	SelectKBest、RFE、LassoCV，用 5-fold CV 比較，再用 1-SE 規則推薦
模型評估	✅	MAE、MSE、RMSE、R²、Adjusted R²、殘差診斷
預測圖加信賴區間 / 預測區間	✅	網頁 Evaluation 分頁、Notebook、報告
CRISP-DM 六步	✅	網頁 1～6 分頁、Notebook、報告
GPT 對話 PDF	✅	`docs/GPT_conversation.pdf`，報告附錄 A
NotebookLM 摘要 100 字以上	✅	475 字；報告第 9 節、網頁第 7 分頁
標示「GPT 輔助內容」「NotebookLM 摘要」	✅	報告和網頁都有這兩個標題
主要繳交：GitHub + Web	✅	repo 和 GitHub Pages 都已上線
選做：對話內容、報告書	✅	`docs/` 資料夾
選做：資訊圖表	✅ 剛補	`images/infographic.png`，README 和網頁都有放

這次補上的

1. **資訊圖表**：一張圖整理 CRISP-DM 流程、主要數字、特徵選擇結果、和網路解法的比較，以及三個關鍵發現。
2. **五大回歸假設補齊**：原本少了「等變異性」的正式檢定，現在加上 Breusch-Pagan 檢定。另外修正「殘差平均為 0」的判斷方式：原本 50 Startups 的測試集只有 10 筆，會被誤判成偏離。
3. **README 刪掉打不開的備用網址**：Streamlit Cloud 那條還沒部署，已經移除。

你的 HW2 資料夾也同步更新了。

報告 PDF 裡還沒寫 GitHub 和網頁網址。報告是選做，不影響評分，如果想加，我可以幫你補在 Word 封面，你再轉一次 PDF。

繳交時給老師這兩個網址：

- GitHub：https://github.com/EasyBoy5566/AIoT-HW2
- Web：https://easyboy5566.github.io/AIoT-HW2/

https://github.com/EasyBoy5566/DIC2
為何我這載入資料及那麼快，這要這麼久

執行了 2 個指令 ▸

我看過你的 DIC2 了。兩個網頁的做法完全不同，所以速度差很多：

	DIC2	HW2（目前）
寫法	純 JavaScript：`index.html` 約 40KB，加兩個圖表 / CSV 套件約 225KB	stlite：在瀏覽器裡跑一整個 Python
打開時要下載的東西	約 0.3MB	Python 執行環境，加上 pandas、numpy、scipy、scikit-learn、statsmodels、plotly、pyarrow，大約數十 MB
計算方式	瀏覽器直接用 JS 算，速度很快	Python 在瀏覽器的 WebAssembly 裡跑，比一般 Python 慢好幾倍
資料量	50 筆、398 筆	洪水資料 20,000 筆 × 20 個特徵，還要跑 3 種特徵選擇 × 5-fold CV

簡單說，DIC2 打開時只要下載一個小網頁；HW2 每次打開都要先下載並啟動整套 Python，再用比較慢的方式跑一大堆模型，所以要等很久。

怎麼改

做法 A：比照 DIC2 改寫成純 JavaScript（推薦）

- 打開大約 1～2 秒就能看到結果，和 DIC2 一樣快。
- 功能全部保留：下拉選兩種 problem、自訂 CSV 網址 / 上傳、三種特徵選擇比較、四種模型、預測區間圖、殘差圖、Q-Q 圖、五大假設檢定、即時預測、AI 輔助分頁。
- 回歸的數學（最小平方法、Ridge、Lasso、信賴區間 / 預測區間、VIF、Breusch-Pagan）我會用 JS 自己寫，再拿 Python 版的結果逐一核對數字，確保算出來一樣。
- 缺點：工作量比較大，`app.py` 會變成只在本機或 Streamlit 用的版本。

做法 B：保留 Python，盡量加速

- 可以拿掉一些套件、把資料再減少，但光是 Python 執行環境本身就有一定大小，大概只能從 20～60 秒縮短到 15～30 秒，還是比 DIC2 慢很多。

我建議做法 A：老師說「跟 DIC2 一樣」，兩份作業的網頁用同一種做法也比較一致。要我照做法 A 改寫嗎？

幫我修正成比照 DIC2 改寫成純 JavaScript

Writing the statistics and diagnostics code, then checking it against Python ▸

我會比照 DIC2，把整個網頁改寫成純 JavaScript。回歸的數學全部用 JS 實作，再拿 Python 版逐項核對數字，最後用瀏覽器實測速度和功能。

已更新任務、執行了一個指令 ▸

DIC2 用的是 Chart.js 4.4.7 和 PapaParse 5.4.1，我直接沿用，畫面風格也比照 DIC2。先寫回歸運算的核心程式。

建立了 5 個檔案、已使用 Claude in Chrome（7 個動作）、以及另外 30 ▸ +4

網頁已經比照 DIC2 改寫成純 JavaScript，並上線到 https://easyboy5566.github.io/AIoT-HW2/

在你的 Chrome 實測：

- **洪水資料**：5 萬筆，從打開網頁到跑完全部分析約 **1.5 秒**，原本要 20～60 秒。
- **老師的 50_Startups 網址**：直接從 GitHub 讀取，分析只要 0.01 秒。

改了什麼

- **做法和 DIC2 一樣**：`index.html` 加上 `js/` 資料夾，圖表和讀 CSV 沿用 DIC2 的 Chart.js 和 PapaParse，畫面配色也比照 DIC2。不再需要在瀏覽器裡下載 Python。
- **功能全部保留**：
  - 下拉選單切換兩種 problem，也能貼自訂 CSV 網址或上傳檔案。
  - 7 個分頁照 CRISP-DM 排。
  - 三種特徵選擇比較和 1-SE 規則推薦，四種模型。
  - 預測圖附 95% 信賴區間和預測區間，殘差圖、Q-Q 圖、五大假設檢定。
  - 即時預測、下載預測結果，最後一頁是 NotebookLM 摘要、GPT 輔助內容和資訊圖表。
- **跟 Python 對過答案**：同樣的資料切分下，JS 算出來的結果和 scikit-learn、statsmodels 一致。
  - OLS 係數、預測區間、VIF、各項檢定幾乎完全相同。
  - 只有 LassoCV 排名特徵時，因為收斂精度不同，CV R² 最多差 0.006。
  - 對照表放在 README。
- **手機也能看**：已確認窄螢幕不會左右捲動。原本很長的「排除的欄位」勾選清單也改成可以收合。

GitHub repo 和你的 HW2 資料夾都已更新，README 和截圖也換成新版。`app.py` 保留下來當 Python / Streamlit 版本，不影響網頁。

洪水資料在網頁上的測試 R² 是 0.8434。這是因為 JS 版的隨機切分和 Python 不同，不是算錯；用同一份切分時，兩邊數字一樣。

回覆

Claude 是 AI，可能會出錯。　Opus 5.5　中　自動